EgoQuant: Questions the judges are likely to ask

Short, honest answers with the evidence behind each. File names are relative to the backend repository unless they start with "frontend:". Numbers are those of 20 October 2026; the live system and `docs/judges/04_TESTING_AND_EVIDENCE.md` *hold the current ones.*

A. The product

1. What problem does EgoQuant solve, and for whom? A retail investor cannot judge whether a portfolio is suitable for them, cannot see why it looks the way it does, and cannot prove later what they were told. EgoQuant gives a portfolio that fits the person's finances and behaviour, inside hard limits, with every number explained and independently re-checked, and a record that can show afterwards, unchanged, what was advised. (`docs/judges/01_PROJECT_SUMMARY.md`)

2. Why several agents instead of one big prompt? Separation of duties. Each agent does one job in deterministic, tested code (market data, portfolio optimisation, behaviour, final check); an **independent Risk Engine** can overrule them all; the language model only explains. One prompt cannot give reproducible numbers, cannot be unit-tested, and cannot be audited. The pieces fail separately and fall back separately. (`README.md` architecture; `egoquant/risk_engine.py`)

3. Where is AI used, and where is it not? *Used:* the advisor chat's explanations (OpenAI), the plain-language explanation of a trade check, and one labelled exception, the market briefing's tone label for a headline and a named fund. *Not used:* scoring, the risk category, every limit, the optimiser, the goal simulation (a statistical model, not an LLM), the Risk Engine, and the trade-check verdict. (`docs/SECURITY.md` section 1; `docs/AGENT_D_LIMITATIONS.md`)

4. What can I try in ten minutes? Create an account (or use the demo account we hand over), fill the 25-question profile (26 steps: one question asks for two amounts), play the two-minute Risk Game, pick interests, set a goal, read the report tabs (Limits and Why show every rule), ask the advisor a what-if and a trade question, check a trade on the Trade check page, confirm the plan, seal it on the test chain and verify the sealed report (Verify a report works without an account). Step-by-step: `docs/judges/05_USER_GUIDE_AND_DEMO.md`.

5. What is new about it? (1) A **Risk Game** that counts one third of the attitude to risk without ever pushing past the finances-based cap. (2) **Behaviour that can only tighten**: real trade history and a crash test make a portfolio more careful, never bolder. (3) An **assistant that can run the real engine** (what-if, check one trade against the saved history) but cannot decide or invent a number. (4) **Tamper-evident advice**: a salted on-chain fingerprint with a public verifier. (5) A **goal simulation that is honest about history**: regime-aware sampling, bias corrections, a calibration backtest, and "N in 100 simulated futures" instead of a percentage. (`docs/judges/01_PROJECT_SUMMARY.md` section 8)

B. Suitability and compliance

6. How is the risk category decided? The category (C1 most careful to C5) is the **lower** of two numbers: *capacity* (what the person's finances allow: how much must come back, horizon, savings, share of wealth, income) and *attitude to risk* (two thirds the questionnaire level plus one third the Risk Game level, rounded). "I could not accept any loss" always gives C1. (`docs/RISK_GAME_BLEND.md`; `egoquant/risk_profile/README.md`; `tests/system/test_risk_game_blend.py`, 300 random investors)

7. Why is the game only one third? Is the game valid? It is hypothetical and self-reported, and we have **no psychometric validation** of it, so we kept it a minority share and bounded it: it can move the

attitude by at most a third of the distance, never past the capacity cap, and two switches (`EGOQUANT_RISK_GAME_WEIGHT`, `EGOQUANT_RISK_GAME_MAY_RAISE`) can remove it or stop it raising. A real trade history, when present, is stronger evidence and only tightens. (`docs/judges/06_LIMITATIONS_AND_ROADMAP.md` 2.2)

8. Does it comply with financial regulation? We do **not** claim certification and it is not a licensed service. We built it around the principles any suitability regime shares: know the client, match product risk to client risk, disclose, keep records, give the client rights over their data. The C1 to C5 and R1 to R5 grades follow the grading customary in Chinese wealth-management practice, but **the numeric limits behind them are our placeholders** (a banner on every report says so).
(`docs/judges/02_SECURITY_COMPLIANCE_REPORT.md` section 3)

9. What stops it recommending something unsuitable? Five layers: the category caps (stocks, product risk level R1 to R5, commodity, crypto and cash caps, loss limit in a named stress scenario); a knowledge gate for complex products; Agent B tightening from behaviour; the independent Risk Engine, which recomputes every limit from the weights alone and fails closed on anything unknown; and the report, which shows the limits and their sources. (`egoquant/risk_engine.py`; `tests/system/`)

10. What about vulnerable clients or minors? Not covered. The prototype does not collect age or test capacity to consent; a person who answers "I could not accept any loss" gets only cash-like products. A real service would add identity, age and vulnerability screening: it is on the roadmap.

11. How would a regulator or a client see why a decision was made? The report states how the category was set, every limit with its source, the reasons in fixed templates, the Risk Engine's checks and the model versions; the questionnaire answers are audit-recorded with a version and hash; and a confirmed plan can be sealed so that any later change to the record is detectable. (`egoquant/report.py`; `docs/ANCHORING.md`)

C. The AI assistant

12. How do you stop it inventing numbers? Number grounding: every percentage, "N in 100", money amount and multiplier in a reply must come from the engine's facts, this turn's tool results or the user's own words. If not, the reply is regenerated once with a correction, then replaced by a safe fallback. (`egoquant/advisor/grounding.py`; `tests/test_grounding_redaction.py`)

13. Can it give investment advice or predict prices? It is instructed not to, the trade verdict reaches it only as "within / outside your limits", a **compliance guard** withholds any sentence that reads like a trade instruction or forecast (36 patterns, English and Chinese), and red-teams check all of this: earlier versions passed the live runs kept in `eval/results/` (advisor chat, prompts v2 to v5: 19/19, 19/19, 21/21 and 24/24; trade explanation: 22/24 then 24/24, 24/24 and 51/51 twice). **On 20 Oct the live red-team found a real defect in the current advisor (the trade tool's two size fields; no trade check completed, a safe failure), fixed the same hour as advisor-v11: 41 of 42 cases pass (the one miss is a false positive of our own keyword check), trade explanation 51 of 51.** The guard is pattern-based and cannot catch every phrasing. (`chat.py`; `egoquant/advisor/config.yaml`; `eval/chat_redteam.py`)

14. Can a judge prompt-inject it? Please try. What it can do is bounded: its only tools are `run_what_if` and `check_trade`, with whitelisted arguments (no field for a limit, category or identity) and at most two calls per message; conversation history and news text are treated as data; the facts it explains are HMAC-signed, and another kind of signed token (news, what-if) is never accepted as facts. (`tests/test_chat_what_if.py`; `tests/accounts/test_chat_trade_tool.py`)

15. What personal data reaches OpenAI? Engine facts and aggregates only, plus what the investor types. A **redaction gateway** replaces e-mail addresses, phone numbers, account ids and the signed-in user's own username with placeholders on the way out and restores them in the reply; the first name used for a greeting is a token (`[NAME_1]`). A name typed inside a question is not recognised, and amounts stay, by design. The model never receives balances, holdings lists or raw trades; requests are sent with `store=false`. (`egoquant/advisor/redaction.py`)

16. What if the AI provider is down or out of credit? The report, limits, trade checks and what-if numbers do not depend on it. The chat answers one plain sentence and explanations fall back to templates. (`tests/advisor/test_ai_down.py`)

17. How can it answer a question about a trade "based on my trading log"? Through the `check_trade` tool: it needs a signed-in account, a portfolio built while signed in, the saved IBKR history and a separate consent. The server loads the holdings from the account (never from the request), runs the **same** deterministic trade check as the Trade check page, and returns the verdict with its numbers; nothing is traded, saved or recorded from the chat. (`egoquant/agents/trade_chat.py`)

D. Security and privacy

18. How are users kept apart? No endpoint takes a user id: identity comes only from the verified token. Every database call runs **as the user** so Supabase Row-Level Security applies, and the backend filters again. There is no all-powerful key in the system, and the account store refuses to start with one. Tests try user A against user B on every endpoint, and the RLS proof passed on the real project. (`tests/accounts/`; `supabase/tests/rls_isolation.sql`)

19. What do you store, and for how long? Account identity, a profile name and picture, the questionnaire answers and a saved report if the person chooses, imported trades (date, symbol, side, quantity, price, currency, category, net cash only), consents, an activity log. **Never**: broker credentials, tokens (used once), uploaded files (parsed in memory), chat text. Every row expires after 180 days. (`docs/judges/02_SECURITY_COMPLIANCE_REPORT.md` section 6)

20. Can a user delete their data? Yes: "Delete my data" and "Delete my account". A test fails if any table with a user column is missing from the erase function. The one thing that cannot be erased is a fingerprint already written to a public chain, which reveals nothing without the record and its salt (both erased on our side; the sign-in and the username stay until the account is deleted).
(`supabase/migrations/007_erase_everything.sql`; `tests/accounts/test_migrations.py`)

21. How do you defend against the usual web attacks? Sessions live in HttpOnly cookies (the page never holds a token); sign-in and account calls check the request's Origin (every other changing route of the website runs the same-origin check; the debug route answers 404 in production); every request body is a strict schema; uploads are parsed with `defusedxml` and size-capped; database access goes through typed REST filters with validated ids, no string-built SQL; errors never echo what was sent. Known gap: the content-security policy still allows inline scripts; a per-request nonce is the planned replacement. (`docs/SECURITY.md`)

22. What did you test, and what did you not? About 3,500 automatic tests (hostile inputs, property sweeps over 300 investors and 5,000 trades, mutation checks, concurrency, deploy checks), live red-teams, read-only probes of the deployed system, and a 14-investor reading of every sentence. **Not done:** a third-party penetration test, a load test of the live service (we flooded a local copy with 20 heavy builds at once, question 31), the live red-team of the newest prompt. (`docs/judges/04_TESTING_AND_EVIDENCE.md`)

23. What exactly goes on the blockchain, and why? `EGOQ1` plus a salted SHA-256 fingerprint of the advice record, in a zero-value transaction on the Sepolia **test** network. The record includes the fingerprint

of the exact final report (rendered once at confirm) and the username it was sealed for, so it proves that this report existed unchanged no later than that block's time, for that username; it does not prove the advice was right, and a printed or PDF copy cannot prove itself. Anyone can check a downloaded sealed report on **Verify a report** (no account needed), and a changed number or a changed name is detected in the browser ("Does not match"). The same format would move to a consortium chain such as FISCO BCOS. (`docs/ANCHORING.md`; `docs/ONCHAIN_DECISIONS.md`; `docs/ON_CHAIN_EVIDENCE.md`; `docs/JUDGE_GUIDE.md`)

E. Methods and data

24. How is the portfolio chosen? Agent C selects the allowed products and prices them; Agent A builds about 100 candidates (20 risk levels times 5 styles) inside every limit; Agent B keeps those inside the person's behavioural comfort band; a Monte Carlo simulation on 10,000 paths of real history picks the one with the highest simulated chance of reaching the goal; the Risk Engine re-checks it. (`docs/judges/01_PROJECT_SUMMARY.md` section 6; `docs/judges/03_METHODS_AND_MODELS.md`)

25. Can I trust "68 in 100"? Treat it as a rough, transparent guide, not a probability. It replays about 19 years of real returns with bias corrections; our calibration backtest shows it improves on the plain bootstrap over all dates but the improvement on later dates is within the noise, and for some portfolio types the realised share differs from the predicted by tens of points. The report says so, shows how much the answer changes between the older and newer half of history, and never calls it a forecast. (`eval/results/regime_v2_summary.md`)

26. Which data do you use, and is it licensed? A committed snapshot of 292 tickers from Yahoo Finance to 16 October 2026 (accepted **for the competition only**; a product needs a licensed feed) plus Tiingo and Binance when keys are available; the live service has run on the saved snapshot only since 20 Oct, 13:00 (no live price request). Every report names the source and the last price day, and invented prices are always labelled.

27. What does Agent B actually do? It reads the IBKR Flex report (upload or a one-off token; read-only), measures things like selling in falls and turnover, adds the crash test, and converts them into numeric limits that can only tighten (a lower loss limit, a lower stock share). Weak evidence is ignored below a confidence threshold. (`egoquant/agents/behavioral/README.md`)

28. Why not use machine learning or reinforcement learning to build the portfolio? Because the task is a constrained optimisation with legal-style limits that must hold exactly and be explainable; a constrained numerical optimiser with an independent checker gives that, a learned policy does not. The regime model only decides which kind of month the simulation draws next, is fitted offline, loaded as a hash-checked file and has a fall-back and a kill switch (`egoquant/regime/`); the only other learned parts are the language model that explains (it never sets a number) and the news-tone label (question 3), which never reaches a weight, a limit or a verdict.

F. Engineering and process

29. How was it built, and how do you know it works? By a small team, each member with their own part of the system, their own git worktree and a written tracker. Every change was tested and secret-scanned before it was pushed (fast-forward pushes only); the last complete test run is in `docs/judges/04_TESTING_AND_EVIDENCE.md` (3,538 passed, one known failing timing test of the news page); a read-only `eval/deploy_check.py` checks a deployment (status, headers, CORS, builds, trade checks). We say it plainly because the process is part of the safety story. (`docs/DEPLOY_ENV.md`)

30. Is a result reproducible? Yes. Seeds are fixed, every report carries input hashes and the engine, model and prompt versions, and the same inputs give identical results on the same machine and library versions (tests compare whole reports; last digits can differ between machines, so a goal chance can move by a point or two, and the optional v2 method by more); the sealed record uses a canonical form that does not depend on the machine. (`egoquant/regime/hook.py`; `tests/regime/test_regime_goal.py`)

31. How does it behave under load? A build takes about 5 to 7 seconds on saved prices on the live service (a heavy 60-year goal build 12 to 17 seconds on our development PC); the server is a single process with per-route rate limits. One portfolio is built at a time per server process (default 1); the others wait their turn. Through the website the wait ends after 115 s with the website's own "too slow" answer; a direct API call gets, after 120 s, a 503 "The service is busy building other portfolios" with Retry-After 30. In our worst local test (20 of the heaviest 60-year builds in the same second) 8 to 11 were served and the rest turned away; without the gate the same test killed the server process in 3 of 4 runs, and no gated run crashed; ordinary builds were not flood-tested. A heavy build alone peaks at about 300 to 415 MB (10 years) and up to 875 MB (30 years) on our development PC, and each extra simultaneous build adds about 150 MB; we did not measure this on the live 1 GB service. It is a prototype for a handful of simultaneous users, not for thousands. (`docs/judges/04_TESTING_AND_EVIDENCE.md` section 7)

32. What would you do with another month? Institutional calibration of the limits, a validation study of the Risk Game, licensed data, a consortium chain, more brokers and Chinese-language advice, a third-party penetration test and monitoring. (`docs/judges/06_LIMITATIONS_AND_ROADMAP.md` section 6)